Course Project Proposal (Software Development)

Name of Project: Forensic Timeline Consistency and Evidence Constraint Analyzer

Short Description:

I want to build a software system that helps forensic teams and investigators organize and check the consistency of an incident timeline. Users will enter evidence events with their known or estimated time, location, and relationships to other events. The system will represent these events as constraints and determine whether they can all occur together. When conflicts exist, it will identify the specific evidence and constraints causing the conflict and generate alternative timelines that remain consistent with the available evidence.

How I Developed the Idea:

My initial Copilot suggestions focused mainly on NLP and text-analysis applications. I was interested in finding a project that I would be more personally motivated to develop throughout a 14-week semester. I have an interest in forensic analysis, and while exploring additional possibilities, I considered how a computer could help investigators check complex timelines without changing the underlying evidence. I discussed this direction with Copilot again and used its suggestions to refine the idea into a constraint-based timeline analysis system. The final project concept and scope were selected by me based on my interests and the course requirements.

Organizational Requirements:

The system will provide an interface for adding, editing, and removing evidence events. Each event can contain a time or time range, location, duration, and relationships or constraints involving other events. Python will be used to represent the evidence as a constraint graph and timeline. Algorithms will check temporal and location constraints, identify impossible combinations, and search for alternative timelines when multiple solutions are possible. The

final application will provide an interactive visualization of the evidence and detected conflicts.

Expected Advantages:

The system will make complex timeline analysis easier to inspect and compare. Instead of only reporting that a contradiction exists, it will identify the evidence involved, show the constraint that was violated, and present other timelines that remain possible. This allows users to examine different explanations while keeping the original evidence unchanged.

Data and Demonstration:

The project will use two types of data. First, I will create synthetic forensic incident datasets containing observations, movements, phone-record-style events, surveillance timestamps, estimated time ranges, locations, and known contradictions. These controlled datasets will allow me to test whether the algorithms correctly detect conflicts and generate valid alternative timelines. Second, I will use the publicly available National Incident-Based Reporting System (NIBRS) dataset from the U.S. Department of Justice, available through Data.gov. NIBRS contains incident-level information including incident date and time, offense type, location, victims, offenders, relationships, and other incident details. I will transform selected NIBRS records into the event-and-constraint format required by the system and use them to demonstrate timeline organization and constraint analysis. Synthetic data will be used for evidence types and contradiction scenarios that are not available in NIBRS. No private case records will be required.

Special Requirements and Resources:

The project will be developed in Python using Pandas for storing, cleaning, and manipulating evidence-event data; NumPy for time and numerical calculations; NetworkX for representing evidence relationships and traversing the constraint graph, python-constraint for defining and

solving time, location, and ordering constraints; Plotly for interactive timeline and conflict visualizations; and Streamlit for the local web-based interface. datetime will be used for time-range and timestamp handling, and JSON/CSV will be used for importing and exporting evidence data. The system will run on a normal laptop and will not require specialized hardware or proprietary case databases. The project will focus on computational timeline consistency and alternative timeline generation rather than making conclusions about guilt, innocence, or the reliability of real evidence.

I used Copilot to explore additional project directions after reviewing the initial suggestions. I found the forensic-analysis direction more interesting and then used Copilot to refine the concept and determine how it could be implemented.

Prompt 1:

I am interested in forensic analysis and want to develop a 14-week Python software project. Can you suggest a practical forensic software system that uses algorithms, data structures, and constraint checking, does not require private case data, and can be demonstrated using public or synthetic data?

Copilot Response:

A possible project is a Forensic Timeline Consistency and Evidence Constraint Analyzer. The system could allow users to enter evidence events with timestamps, locations, and relationships between events. It could represent these events as constraints and use algorithms to identify impossible sequences, conflicting evidence, and alternative timelines that satisfy the available constraints. Synthetic data could be used to test known contradictions, while publicly available crime and incident datasets could provide additional data for demonstration.

Prompt 2:

How can I obtain realistic data for this forensic timeline project without using private investigation records? Are there publicly available datasets that contain incident dates, times, locations, offenses, victims, offenders, or related information that could be transformed into timeline events?

Copilot Response:

A suitable source is the National Incident-Based Reporting System (NIBRS), which provides publicly available incident-level crime data through U.S. government data resources. The data includes information such as incident dates and times, offense types, locations, victims, offenders, and relationships. These records could be transformed into the event and constraint format required by the application. Synthetic data can supplement the public data for testing specific contradictions, uncertain time ranges, and alternative timeline scenarios.